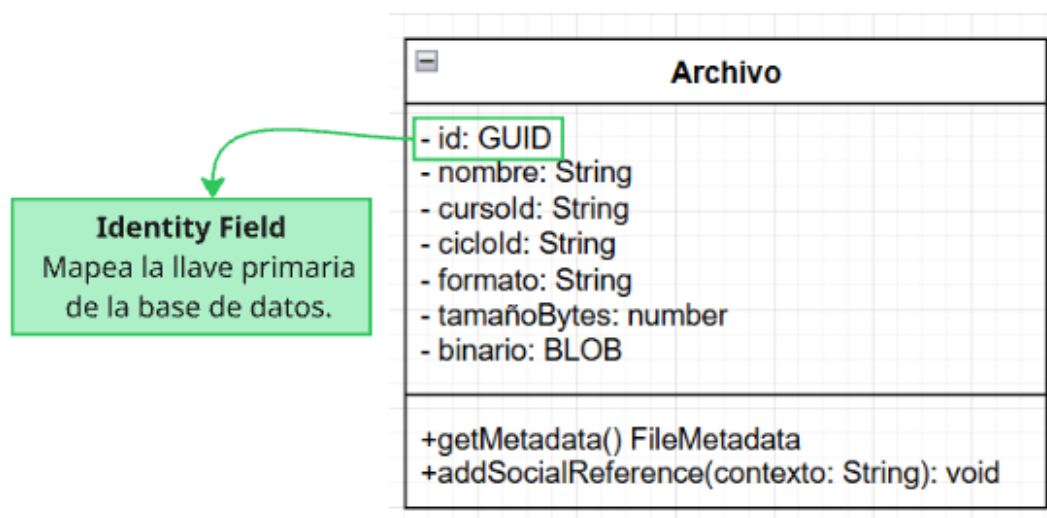


Patrones de Persistencia e Interacción con BD

Patrones de Persistencia:

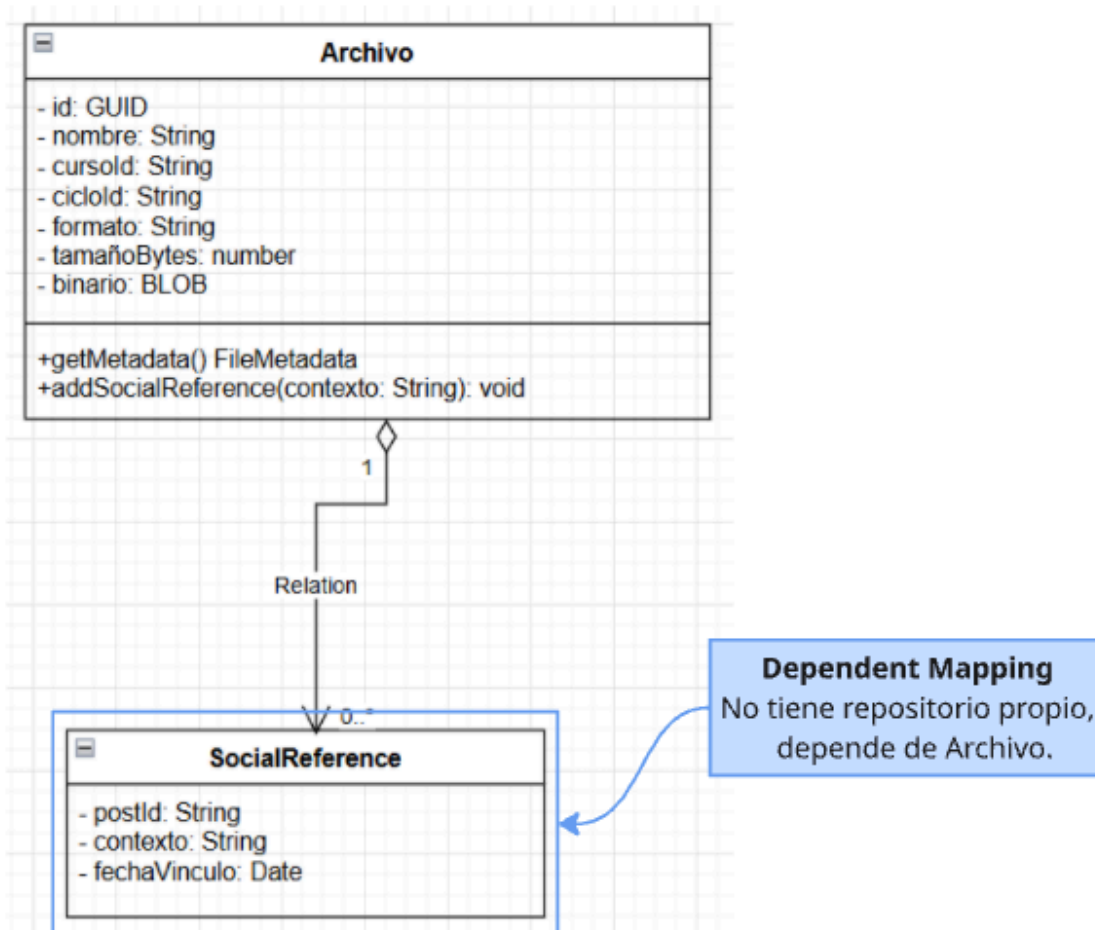
- **Identity Field:**

Para mantener la relación unívoca entre los objetos en memoria y los registros en la base de datos/Hazelcast, se implementó el patrón Identity Field. La clase Archivo incluye el atributo id: GUID. Esto simplifica enormemente las operaciones CRUD en la clase HazelcastFileRepository (como findById), ya que el identificador es inmutable, representativo y permite localizar el registro exacto en la grilla de datos sin colisiones.



- **Dependent Mapping:**

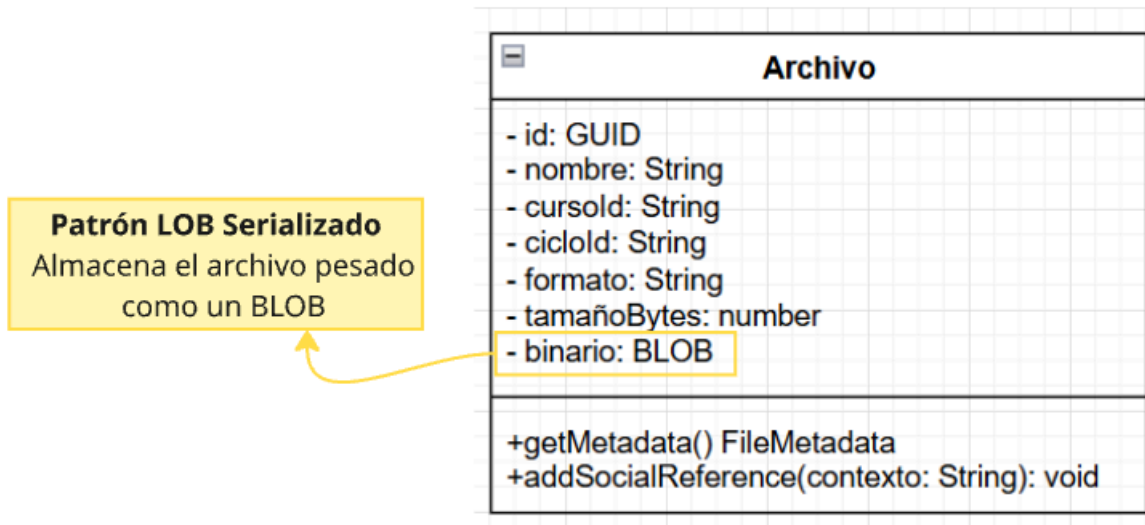
Se identificó una relación de composición estricta entre un documento y los foros donde se comparte. Para esto, se aplicó el patrón Dependent Mapping. La clase dependiente SocialReference no tiene su propio ciclo de vida ni un repositorio independiente; su persistencia es gestionada enteramente por la clase "dueña" (Archivo). Cuando el repositorio invoca addSocialLink(), actualiza el LOB del Archivo agregando esta nueva referencia, manteniendo una alta cohesión y evitando la necesidad de crear tablas o repositorios innecesarios.



Patrones de Interacción con BD:

- **Objeto Grande (LOB) Serializado:**

Se aplicó el patrón Objeto Grande (LOB) Serializado en la entidad Archivo. Dado que el sistema debe almacenar documentos pesados (exámenes, apuntes), guardar el contenido físico archivo por archivo en el sistema operativo dificultaría la gestión distribuida. Por ello, el atributo - binario: BLOB serializa el archivo en formato binario para almacenarlo directamente en la base de datos/grilla de datos junto con sus metadatos. Esto garantiza la consistencia transaccional (el archivo y sus datos como autor o curso se guardan en una misma transacción) y facilita su distribución en nuestra arquitectura orientada a espacios.



- **Data Mapper y Data Transfer Object (DTO):**

Para aislar la estructura interna de la base de datos de la capa de presentación, se implementó el patrón Data Mapper junto con el patrón DTO. La clase ArchivoMapper es la única responsable de transferir datos bidireccionalmente entre el repositorio (entidad Archivo) y el cliente (ArchivoDTO). Esto protege la información sensible (como el buffer binario) enviando por la red únicamente los datos serializables y necesarios para la vista (como urlDescarga). Al independizar estas estructuras, si la tabla de archivos cambia en el futuro, solo se modifica el Mapper sin afectar el resto de la aplicación web.

